

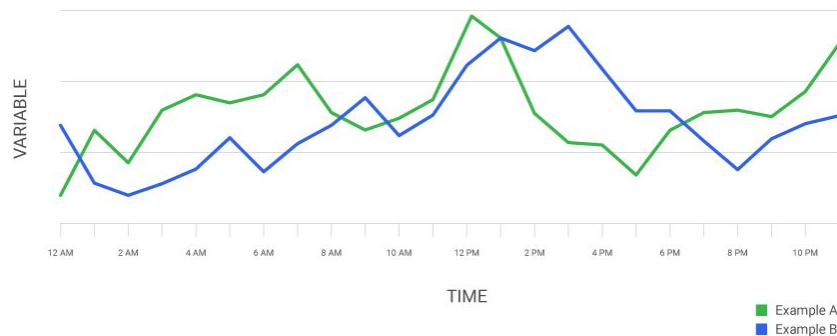
Kaggle - Time Series II #12



Recap : Basics

What is time series:

- A set of observations recorded over time (i.e daily/monthly..)

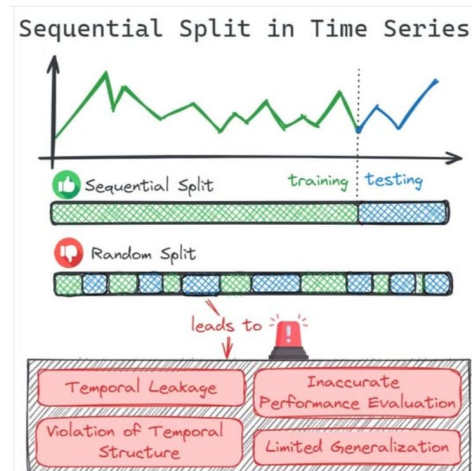
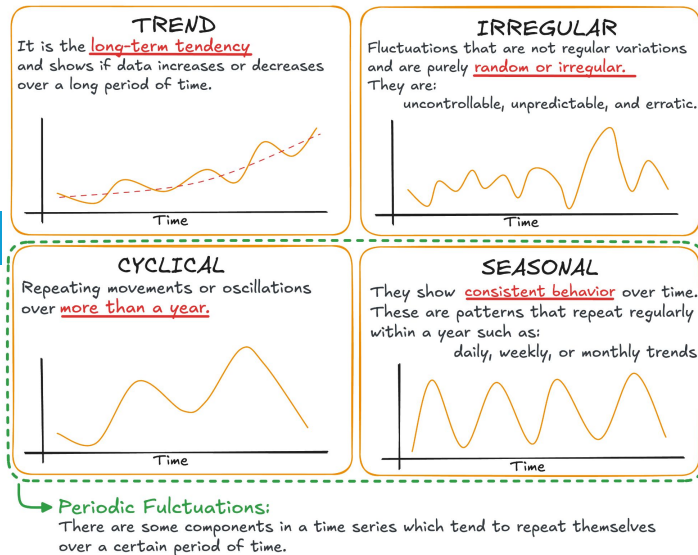


	date	value
1	2023-01-01	145
2	2023-02-01	212
3	2023-04-01	265
4	2023-05-01	299
5	2023-06-01	345
6	2023-09-01	278
7	2023-10-01	256
8	2023-11-01	202
9	2023-12-01	176

Forecasting : predicting a value in the future

Recap

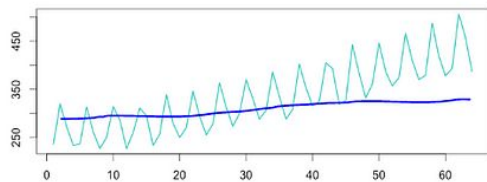
- Seasonality , trends , cyclicity and noise
- Lag features
- Window features
- Extracting date and time features
- Data preprocessing tips - regular intervals
- Data Leakage
- Simple Moving Average



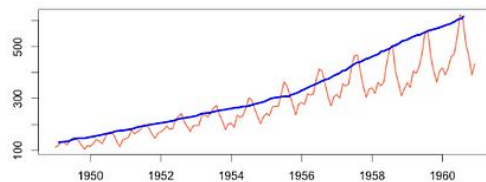
Time Series Decomposition

- **Level:** The average value in the series.
- **Trend:** The increasing or decreasing value in the series.
- **Seasonality:** The repeating short-term cycle in the series.
- **Noise:** The random variation in the series.

- **Additive:** $y(t) = \text{Level} + \text{Trend} + \text{Seasonality} + \text{Noise}$
- **Multiplicative:** $y(t) = \text{Level} * \text{Trend} * \text{Seasonality} * \text{Noise}$



Australian beer production – The seasonal variation looks constant; it doesn't change when the time series value increases. We should use the **additive model**.



Airline Passenger Numbers – As the time series increases in magnitude, the seasonal variation increases as well. Here we should use the **multiplicative model**.

Additive:
Time series = Seasonal + Trend + Random

Multiplicative:
Time series = Trend * Seasonal * Random

Why do we care if its additive or multiplicative - is it that deep?

- We want to be able to model the underlying components of a time series like trend, seasonality and noise , because it makes us better at forecasting / predicting future values.

Lags

- Time series lags represent past values of a variable and are used to predict current/ future values
- Capture temporal dependencies and trends

t	value	lag1 value(t-1)	lag2 value(t-2)
1	5	NaN	NaN
2	10	5	NaN
3	15	10	5

```
df['silver_lag_1'] = df['silver'].shift(1)  
df['silver_lag_5'] = df['silver'].shift(5)
```

Chronological order and equidistant time stamps

```
import pandas as pd
# Convert to datetime if necessary
df['date'] = pd.to_datetime(df['date'])
# Sort chronologically
df = df.sort_values(by='date')
```

```
# Check time intervals (delta)
df['delta'] = df.index.to_series().diff()
print(df['delta'].value_counts()) # Should be consistent
```

Filling in missing values

1. Fill NaN with mean value
2. Fill Nan with last value with `.ffill()`
3. **Fill NaN with Linearly Interpolated value** with `.interpolate()` = method for generating points between given points.

Before

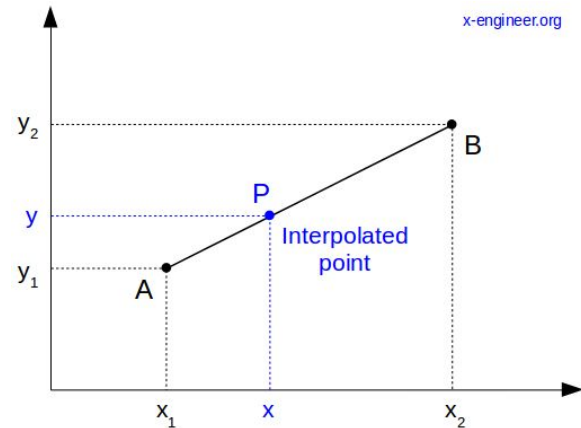
Number of days
1.0
2.0
NaN
4.0
5.0
NaN
6.0

After 'ffill'

Number of days
1.0
2.0
2.0
4.0
5.0
5.0
6.0

After 'bfill'

Number of days
1.0
2.0
4.0
4.0
5.0
6.0
6.0



Stationarity

What is it

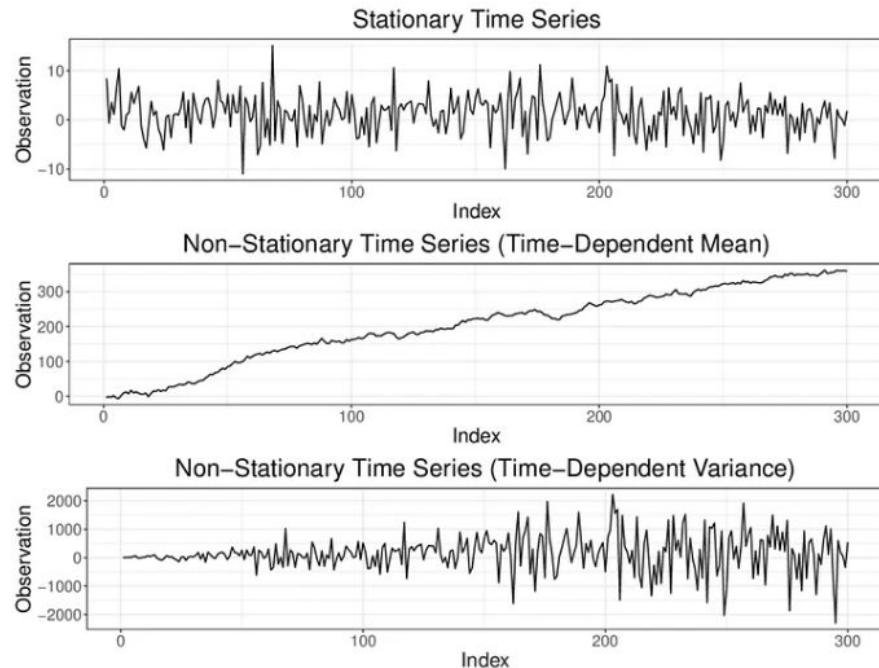
- A dataset's mean and variance are constant / do not change over time

How to detect?

1. Visually
2. Statistical tests

Why do I care?

- Better forecasting accuracy
- assumption



Augmented Dickey Fuller Test (ADF)

A hypothesis test used to determine if a time series is stationary

H0: “the series is non stationary”

H1: “the series is stationary”

A small p value ($p < 0.05$) -> reject H0 -> indicative that the data is stationary

Augmented Dickey Fuller Test (ADF)

```
1 from pandas import read_csv
2 from statsmodels.tsa.stattools import adfuller
3 series = read_csv('daily-total-female-births.csv', header=0, index_col=0,
4 X = series.values
5 result = adfuller(X)
6 print('ADF Statistic: %f' % result[0])
7 print('p-value: %f' % result[1])
8 print('Critical Values:')
9 for key, value in result[4].items():
10     print('\t%s: %.3f' % (key, value))
```

```
1 ADF Statistic: -4.808291
2 p-value: 0.000052
3 Critical Values:
4     5%: -2.870
5     1%: -3.449
6     10%: -2.571
```

Do we reject H_0 ? Is the data stationary or non-stationary?

Data is stationary as $0.000052 < 0.05$

How to transform data so its stationary

1. Transformation : basically some sort of maths operation like “log” or “square root”

```
# Transformation (e.g., Logarithmic)  
ts_log = np.log(ts)
```

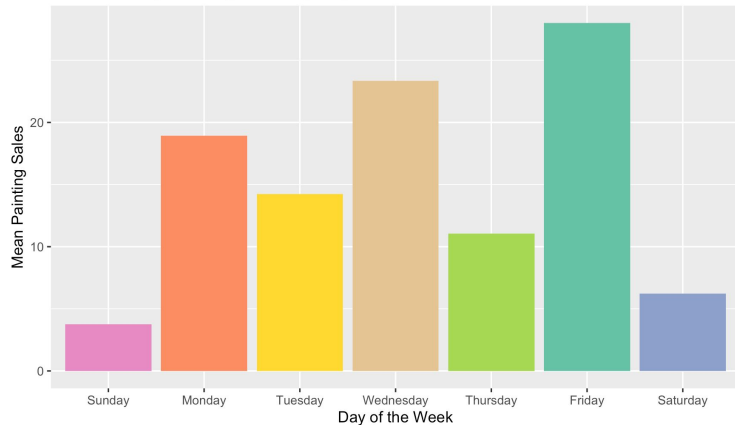
2. Differencing: when you compute the difference between consecutive observations

$$y'_t = y_t - y_{t-1}$$

Extracting date and time features

```
train['year'] = train['date'].dt.year  
train['month'] = train['date'].dt.month  
train['day'] = train['date'].dt.day  
train['dayofweek'] = train['date'].dt.dayofweek
```

Mean Weekly Painting Sales by Day of the Week

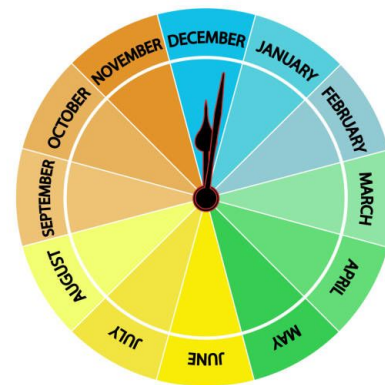


- Allows us to spot patterns better -> better forecasting

Encoding cyclical features

When encoding cyclical time data (like months 1-12), ML models incorrectly interpret December (12) and January (1) as being far apart, when they're adjacent in a cycle

Circular encoding preserves this circular continuity of time → this enables models to learn recurring patterns naturally



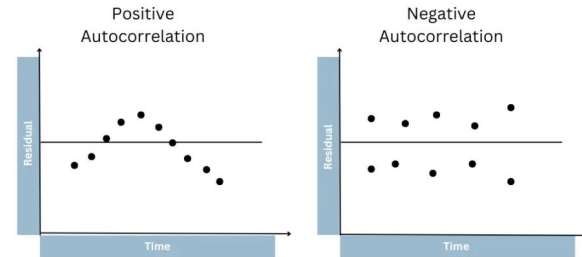
Encoding cyclical features

- You can use sin, cosine functions to interpret these months on a unit circle vs as a linear number line

```
month_in_year = 12
df['month_sin'] = np.sin(2*np.pi*df['month']/month_in_year)
df['month_cos'] = np.cos(2*np.pi*df['month']/month_in_year)
```

Autocorrelation

- Measures the relationship between a variable's current value and its past values (lagged versions) over time
- Positive autocorrelation: a high value today suggests a high value tomorrow
- Negative autocorrelation: a high value today suggests a low value tomorrow



Autocorrelation Analysis - ACF

- AutoCorrelation Function (ACF) = statistical method to quantify autocorrelation for a range of lags

Formula for ACF

The ACF at lag k for a time series x_t is given by:

$$ACF(k) = \frac{\sum_{t=k+1}^T (x_t - \bar{x})(x_{t-k} - \bar{x})}{\sum_{t=1}^T (x_t - \bar{x})^2}$$

Here, T is the total number of observations, and $\bar{x}$ is the mean of the time series.

- ACF = 1:
Positive autocorrelation
- ACF = -1:
Negative autocorrelation

Types of Time series models

- Univariate
 - ARIMA
 - Prophet
- Multivariate
 - LGBM
 - Linear Regression

Summary

- Time series decomposition : additive + multiplicative
- Lags
- Filling in missing values
- Stationarity + ADF test
- Extracting date and time features
- Encoding Cyclical features
- Autocorrelation + ACF
- Types of Time series models
- Note this is just a snapshot of time series stuff! Go through these slides and explain / research more into these topics

Research task

1. Get into pairs
 2. Research a model:
 - a. ARIMA
 - b. Exponential Smoothing
 - c. Prophet
 - d. Linear Regression
 3. You have 25 minutes to research this model
 - a. What is it?
 - b. Why is it useful / applications?
 - c. Key components/ maths behind it?
 - d. **Challenge** : small code snippet/ implementation
 4. Present at the end in under 5 minutes: (you can use 1-2 slides or just speak about it / use pen paper)
- Some resources
 - DataCamp
 - TowardsDataScience

Kaggle competition

- Old one but good for time series:
<https://www.kaggle.com/competitions/playground-series-s3e19/data>
- Current one:
<https://www.kaggle.com/competitions/qmml-time-series/data>

Feedback Form

We will look at
deep learning
after reading
week.

